

Workshop 1 - 03docker

```
feijo@Valenfei MINGW64 ~/03-docker/WS1 (master)
$ mkdir api

feijo@Valenfei MINGW64 ~/03-docker/WS1 (master)
$ mkdir nginx

feijo@Valenfei MINGW64 ~/03-docker/WS1 (master)
$ cat > api/app.py << 'EOF'
import os
from flask import Flask, jsonify

app = Flask(__name__)
INSTANCE = os.getenv("INSTANCE", "api-local")

@app.get("/")
def home():
    return jsonify({
        "message": "Hola desde Flask",
        "instance": INSTANCE
    })

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
EOF

feijo@Valenfei MINGW64 ~/03-docker/WS1 (master)
$
```

```
feijo@Valenfei MINGW64 ~/03-docker/WS1 (master)
$ cat > api/requirements.txt << 'EOF'
Flask
EOF

feijo@Valenfei MINGW64 ~/03-docker/WS1 (master)
$ cat > api/Dockerfile << 'EOF'
FROM python:3.12-slim

WORKDIR /app

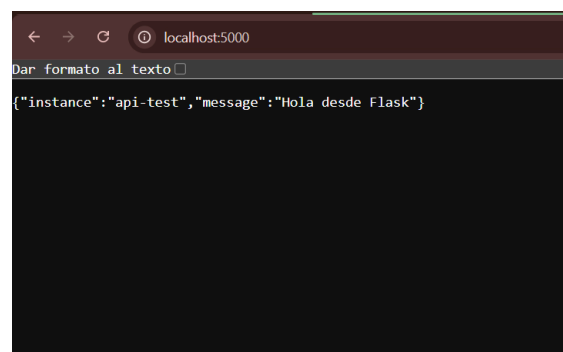
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]
EOF

feijo@Valenfei MINGW64 ~/03-docker/WS1 (master)
$
```



localhost:5000

Dar formato al texto

```
{\"instance\": \"api-test\", \"message\": \"Hola desde Flask\"}
```

```
feijo@Valenfei MINGW64 ~/03-docker/WS1 (master)
$ cat > nginx/nginx.conf << 'EOF'
upstream flask_backend {
    server api1:5000;
    server api2:5000;
    server api3:5000;
}

server {
    listen 80;

    location / {
        proxy_pass http://flask_backend;
    }
}
EOF

feijo@Valenfei MINGW64 ~/03-docker/WS1 (master)
$ cat > docker-compose.yml << 'EOF'
services:
  api1:
    build: ./api
    environment:
      INSTANCE: api-1

  api2:
    build: ./api
    environment:
      INSTANCE: api-2

  api3:
    build: ./api
    environment:
      INSTANCE: api-3

  nginx:
    image: nginx:alpine
    ports:
      - "8080:80"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - api1
      - api2
      - api3
EOF
```

```

feijo@Valenfei MINGW64 ~/03-docker/WS1 (master)
$ docker compose up --build
Image wsl-api1 Building
Image wsl-api2 Building
Image wsl-api3 Building
1 [internal] load local bake definitions
1 reading from stdin 1.36kB 0.0s done
1 DONE 0.0s
2 [api1 internal] load build definition from Dockerfile
2 transferring dockerfile: 205B done
2 DONE 0.0s
3 [api3 internal] load metadata for docker.io/library/python:3.12-slim
3 DONE 0.9s
4 [api3 internal] load .dockerignore
4 transferring context: 2B done
4 DONE 0.0s
5 [api3 internal] load build context
5 transferring context: 63B done
5 DONE 0.0s
6 [api1 1/5] FROM docker.io/library/python:3.12-slim@sha256:09f7da3bc104798d0aff
6 resolve docker.io/library/python:3.12-slim@sha256:09f7da3bc104798d0aff
6 DONE 0.0s
7 [api3 2/5] WORKDIR /app
7 CACHED

```

```

feijo@Valenfei MINGW64 ~/03-docker/WS1 (master)
$ for i in {1..12}; do curl -s http://localhost:8080/; echo; done
{"instance":"api-1","message":"Hola desde Flask"}
{"instance":"api-1","message":"Hola desde Flask"}
{"instance":"api-1","message":"Hola desde Flask"}
{"instance":"api-1","message":"Hola desde Flask"}
{"instance":"api-1","message":"Hola desde Flask"}
{"instance":"api-1","message":"Hola desde Flask"}
{"instance":"api-1","message":"Hola desde Flask"}
{"instance":"api-1","message":"Hola desde Flask"}
{"instance":"api-1","message":"Hola desde Flask"}
{"instance":"api-1","message":"Hola desde Flask"}
{"instance":"api-1","message":"Hola desde Flask"}
{"instance":"api-1","message":"Hola desde Flask"}

```

Ahora api2 no aparece en las respuestas

```

feijo@Valenfei MINGW64 ~/03-docker/WS1 (master)
$ docker compose stop api2
Container wsl-api2-1 Stopping
Container wsl-api2-1 Stopped

feijo@Valenfei MINGW64 ~/03-docker/WS1 (master)
$ for i in {1..12}; do curl -s http://localhost:8080/; echo; done
{"instance":"api-3","message":"Hola desde Flask"}
{"instance":"api-3","message":"Hola desde Flask"}
{"instance":"api-1","message":"Hola desde Flask"}
{"instance":"api-3","message":"Hola desde Flask"}
{"instance":"api-1","message":"Hola desde Flask"}
{"instance":"api-3","message":"Hola desde Flask"}
{"instance":"api-1","message":"Hola desde Flask"}
{"instance":"api-3","message":"Hola desde Flask"}
{"instance":"api-1","message":"Hola desde Flask"}
{"instance":"api-3","message":"Hola desde Flask"}
{"instance":"api-1","message":"Hola desde Flask"}
{"instance":"api-3","message":"Hola desde Flask"}

```

Reto

```

feijo@Valenfei MINGW64 ~/03-docker/WS1 (master)
$ cat > api/app.py << 'EOF'
import os
from flask import Flask, jsonify

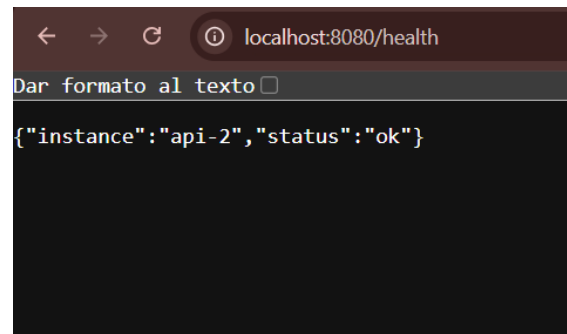
app = Flask(__name__)
INSTANCE = os.getenv("INSTANCE", "api-local")

@app.get("/")
def home():
    return jsonify({
        "message": "Hola desde Flask",
        "instance": INSTANCE
    })

@app.get("/health")
def health():
    return jsonify({
        "status": "ok",
        "instance": INSTANCE
    })

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
EOF
feijo@Valenfei MINGW64 ~/03-docker/WS1 (master)
$

```



localhost:8080/health

Dar formato al texto

```
{\"instance\": \"api-2\", \"status\": \"ok\"}
```

Preguntas

1. ¿Por qué el cliente solamente conoce `localhost:8080` ?

Porque Nginx actúa como punto de entrada único (reverse proxy). El cliente nunca se conecta directamente a `api1` , `api2` o `api3` — esos nombres solo existen dentro de la red interna de Docker Compose. Esto genera bajo acoplamiento: el cliente no necesita saber cuántas instancias existen ni dónde están ubicadas.

2. ¿Qué función cumple `upstream` ?

Define un grupo de servidores backend (en este caso `api1:5000` , `api2:5000` , `api3:5000`) que pueden atender el mismo tipo de solicitud. Nginx usa ese grupo como pool de destinos posibles cuando llega una petición a `proxy_pass http://flask_backend;` .

3. ¿Qué estrategia utiliza Nginx por defecto en este escenario?

Round Robin: distribuye las solicitudes de forma secuencial y cíclica entre los servidores del `upstream` (`api1` → `api2` → `api3` → `api1` → ...), ya que no se configuró ninguna estrategia adicional (ni `least_conn` , ni `ip_hash` , ni pesos).

4. ¿Qué ocurre al detener `api2` ?

Nginx deja de enviarle tráfico y las solicitudes se siguen repartiendo entre `api1` y `api3` . El servicio como un todo sigue disponible para el cliente, aunque con menor capacidad total — esto ilustra tolerancia a fallos parcial gracias a tener múltiples instancias.

5. ¿Qué componente podría convertirse en un punto único de falla?

Nginx mismo. Si el contenedor de Nginx se cae, el cliente pierde el único punto de entrada y no puede llegar a ninguna instancia de Flask, aunque `api1`, `api2` y `api3` sigan funcionando perfectamente.

6. ¿Por qué este diseño representa escalamiento horizontal?

Porque para aumentar la capacidad del sistema se agregan más instancias iguales de la misma aplicación (más contenedores Flask), en lugar de darle más recursos (CPU/RAM) a una sola instancia (que sería escalamiento vertical). El balanceador permite que esas instancias adicionales se integren de forma transparente para el cliente.

7. ¿Qué tendría que cambiar para tener cinco instancias?

- Agregar `api4` y `api5` en `docker-compose.yml`, cada uno con su propio `INSTANCE` (`api-4`, `api-5`) y `build: ./api`.
- Agregarlos también al bloque `upstream` en `nginx.conf` (`server api4:5000;` y `server api5:5000;`).
- Agregarlos a `depends_on` del servicio `nginx`.
- Reconstruir con `docker compose up --build`.